

업무일지 — 이진호 (개인 프로젝트: RFP RAG 시스템)

2026-06-18 (금) · 프로젝트 이해 · 개인 전환 · 스택 결정

오늘 할 일

- 중급 프로젝트가 뭘 요구하는지 파악
- 개인 프로젝트 전환에 따른 진행 방식 확인
- 기술 스택 결정, 레포 셋업

한 일 / 느낀 것

- 팀용과 개인용 가이드가 다르다는 걸 확인. 개인은 클라우드 서버가 없어서 로컬 환경에 LLM API를 붙이는 방식으로 진행. 발표는 없고 결과물은 깃허브 레포, 보고서, 업무일지.
- 자체 서빙 방식과 API 방식을 비교해봤다. 자체 서빙은 데이터를 밖으로 안 보내는 대신 GPU와 운영 부담이 크고, API는 쉽고 품질이 좋지만 데이터가 외부로 나간다. 혼자 하는 거라 API로 완성도부터 잡고, 자체 서빙은 선택사항으로 남기기로.
- 벡터 DB는 로컬 저장과 메타데이터 필터가 되는 걸로 잠정 결정. 임베딩과 생성 모델은 과제에서 허용된 것 사용.
- 레포 만들고 디렉토리 구조 스케치.

고민 / 실수

- API 키를 어떻게 조달할지 애매. 개인 전환이라 팀 키를 못 받는 상황이라 우선 내 키로 진행하기로. 키는 환경설정 파일로 분리하고 커밋에 딸려가지 않게 하는 습관부터 잡음.

소감

- 혼자 다 해야 해서 막막하지만, 범위를 스스로 정할 수 있는 건 오히려 자유롭다. 일단 작게라도 돌아가는 것부터 만들자는 마음.

2026-06-19 · 데이터 확보 · 탐색

오늘 할 일

- 데이터 내려받고 구조 파악
- 어떤 필드가 검색과 필터에 쓸모 있는지 확인

한 일 / 인사이트

- 문서 100건(대부분 한글파일, 일부 PDF)과 메타데이터 파일 확인. 공고번호, 사업명, 사업금액, 발주기관, 마감일, 사업요약에 본문 텍스트 컬럼까지 있다.
- 핵심 발견: 메타데이터에 본문이 이미 어느 정도 추출돼 있다. 한글파일 파싱을 일단 건너뛰고 이걸로 베이스라인을 빨리 만들 수 있겠다. 다만 문서당 몇천 자 수준이라 원문 전체가 아니라 일부다. 나중에 문제될 수 있으니 메모해둬.
- 발주기관과 사업명을 필터로 쓰면 여러 문서 사이에서 타깃을 좁힐 수 있겠다는 아이디어.

고민

- 공고번호에 결측치가 꽤 있다. 문서 식별자로 쓰려면 유일성 처리가 필요하겠다. (이때는 가볍게 넘겼는데 며칠 뒤 버그로 돌아옴)

소감

- 열어보니 진짜 실무 RFP라 흥미롭다. 한 문서가 수십 페이지라, 왜 이것 요약하고 검색해주는 시스템이 필요한지 체감된다.

2026-06-22 · 베이스라인 파이프라인 구현 · 첫 버그

오늘 할 일

- 로딩부터 청킹, 임베딩, 검색, 생성까지 일단 돌아가는 파이프라인 만들기

한 일

- 로딩, 청킹, 벡터 저장, 검색, 생성으로 모듈을 나눠 구현했다. 청킹은 문단 경계를 존중하게 잡고, 생성 프롬프트에는 근거 기반으로만 답하고 없는 내용은 모른다고 말하라는 규칙을 넣어 환각을 억제했다.

고민 / 실수 (중요)

- 첫 인덱스 구축에서 중복 ID 에러가 터졌다. 공고번호가 빈 문서들의 ID가 전부 같은 값이 되어 저장에 실패한 것. 그저께 "나중에 처리해야지" 했던 결측치가 그대로 사고로 돌아왔다. 공고번호가 없으면 행 번호를 쓰고 겹치면 접미사를 붙여 무조건 유일하게 만들어 해결.
- 교훈: ID 유일성은 미루지 말고 처음부터 보장할 것.

결과

- 버그 잡은 뒤 국민연금공단 이러닝시스템 질문에 정답 문서를 찾아 근거와 함께 답변. 베이스라인이 끝까지 돌아왔다.

소감

- 처음 돌아가는 걸 봤을 때 짜릿했다. 근데 답이 좀 얇은 느낌. 본문이 짧아서 그런가 싶다. 정량 평가를 붙여야 진짜 상태를 알 수 있겠다.
-

2026-06-23 · 평가 설계

오늘 할 일

- 느낌이 아니라 숫자로 성능을 볼 수 있게 평가 체계 만들기

한 일 / 인사이트

- 가이드의 공식 예시 질문에 기관명을 부정확하게 쓴 케이스와 없는 정보를 묻는 케이스(환각 체크)를 더해 10문항 평가셋을 만들었다. 각 질문에 정답 발주기관을 라벨로 붙여 검색을 객관적으로 채점할 수 있게 했다.
- 지표는 직접 설계했다. 검색 커버리지, 첫 정답 순위, 답변의 근거성과 관련성, 없는 정보에 대한 거절 정확도, 응답 속도.
- 근거성과 관련성은 정답을 일일이 손으로 달 수 없으니, 심판 역할 LLM에게 근거와 답변의 정합성을 채점시키기로.

고민

- 후속질문은 턴을 순서대로 실행하고 마지막 답을 평가하는 방식으로. LLM 심판을 얼마나 믿을 수 있을지 걱정되지만, 설정끼리 상대 비교하는 용도로는 충분하다고 판단.

소감

- 평가셋을 만들다 보니 이 시스템이 실제로 뭘 잘해야 하는지가 선명해졌다. 단일 추출, 다중 종합, 후속 맥락, 환각 방지 — 이 네 가지구나.
-

2026-06-24 · 평가 자동화 · 심판 버그 · 검색 약점 발견

오늘 할 일

- 평가 자동화 마무리하고 베이스라인 점수 측정

한 일 / 결과

- 베이스라인을 재보니 검색이 약점이라는 게 숫자로 드러났다. 정답 문서를 평균 절반 정도밖에 못 찾아온다. 특히 두 기관을 비교하는 질문은 한쪽만 걸리고, 기관명을 부정확하게 쓰면 아예 못 찾는다.

- 환각 방지는 잘 된다. 없는 사업엔 지어내지 않고 모른다고 답한다.

고민 / 실수 (중요)

- 심판 점수가 전부 비어 나와서 한참 헤맸다. 원인은 모델 특성이었다. 쓰고 있는 생성 모델이 추론 형이라 출력 토큰을 추론에 먼저 써버려서, 토큰을 뽁뽁하게 주면 본문이 빈 채로 돌아온다. 토큰을 넉넉히 주고 출력 파싱도 실패를 가정하고 방어적으로 바꿔 해결했다. 리랭커 쪽에도 같은 함정이 있어서 같이 고침.
- 교훈: 추론형 모델은 출력 토큰을 넉넉히 주고, LLM 출력 파싱은 항상 실패를 가정할 것.

소감

- 숫자가 마음을 편하게도, 불편하게도 만든다. 잘 되는 줄 알았는데 절반이라니. 그래도 약점을 특정했으니 방향은 분명하다.

2026-06-25 · 검색 기법 비교 실험

오늘 할 일

- 여러 검색 설정을 같은 평가셋으로 돌려서 뭘 채택할지 결정

한 일 / 의사결정

- 단순 유사도, 다양성을 주는 검색(MMR), 후보 수 확대, 기관명 자동필터, LLM 리랭크를 비교했다.
- 자동필터가 검색 부문 최고였다. 질문 속 기관명을 느슨하게 매칭해서 발주기관 필터를 거니 정답 문서가 1순위로 올라오고 속도도 제일 빨랐다 → 채택.
- LLM 리랭크는 폐기. 검색 지표는 그대로인데 시간만 두 배로 든다. 안 되는 실험도 근거로 남긴다.
- 검색 후보 수를 조금 늘리는 건 커버리지에 도움이 돼서 자동필터와 결합하기로.

여기서 드러난 더 큰 두 문제

1. 답변 품질이 전반적으로 낮다. 검색을 잘해도 메타데이터 본문이 얇아서 세부 요구사항이 없다 → 원본 파일 전체 파싱으로 전환하기로 결정.
2. 후속질문이 전멸. "더 알려줘" 같은 질문엔 기관명이 없어서 검색이 샌다 → 대화 맥락을 반영해 질문을 다시 쓰는 기능을 넣기로 결정.

소감

- 실험이 다음에 뭘 해야 할지를 정확히 가리켜줬다. 안 되는 실험을 근거로 남긴 게 오히려 값지다.
-

2026-06-26 ~ 06-30 · 원본 파싱 · 쿼리 재작성 · 개선 검증

오늘 할 일

- 원본 파일 전체 파싱 도입, 후속질문 재작성 추가
- 원본으로 재인덱싱해서 품질이 오르는지 검증

한 일

- 한글파일 파서 개선. 처음 만든 버전은 본문에 깨진 글자 잡음이 섞였는데, 파일 포맷의 제어 문자를 안 건너뛴 게 원인이었다. 고치고 나니 문서당 글자 수가 메타데이터 본문의 스무 배 가까이 나왔고, 100건 전부 파싱에 성공했다.
- 대화 히스토리를 반영해 후속질문을 독립 질문으로 다시 쓴 뒤 검색하는 기능을 추가했다.

고민 / 실수 (원본 인덱싱 크래시)

- 원본 재인덱싱 중에 인코딩 에러로 크래시가 났고, 인덱스가 텅 빈 채로 남았다. 그걸 모르고 실험을 돌렸더니 커버리지는 전부 0인데 근거성 점수는 높게 나오는 이상한 결과가 나왔다. 뜯어보니 빈 인덱스라 매번 모른다고 답했고, 심판이 그 거절을 "환각이 없으니 충실하다"로 보상한 착시였다.
- 교훈: 근거성이 높아도 커버리지가 0이면 잘한 게 아니라 아무것도 안 한 것이다. 지표는 항상 교차검증할 것.
- 근본 원인은 한글파일 원문 속 특수문자를 파서가 잘못 쪼갰 것이었다. 결합 처리와 인코딩 안전장치를 더하고, 전 문서가 통과하는 걸 확인한 뒤 재빌드했다.

고민 / 실수 (재실행 전 선제 점검)

- 재빌드하니 이번엔 벡터 DB의 한 번에 저장할 수 있는 상한에 걸렸다. 원본 체크 수가 확 늘어나서 터진 것 → 나눠서 저장하도록 수정.
- 같은 실수를 반복하기 싫어서, 재실행 전에 API 없이 로컬에서 돌릴 수 있는 부분(저장, 파싱, 필터)을 전부 미리 점검했다. 덕분에 자동필터의 숨은 결함 둘을 선제 발견했다. 비교 질문에서 한 기관만 잡고 나머지를 걸러내는 문제, 그리고 기관명 일부가 우연히 겹쳐 엉뚱한 기관이 딸려오는 문제. 매칭 기준을 더 엄격하게 하고, 여러 기관이 잡히면 모두 포함하도록 고쳤다.
- 교훈: 실패를 한 번 겪었으면, 다음 실행 전에 로컬에서 돌릴 수 있는 건 미리 다 돌려보고 선제적으로 잡자.

소감

- 얇은 데이터를 붙잡고 있던 게 계속 걸렸는데, 원본을 제대로 뽑아내니 속이 시원하다. 다만 "커버리지 0인데 근거성 만점"이라는 착시에 잠깐 속은 게 뜨끔했다. 숫자를 액면 그대로 믿으면 안 된다는 걸 몸으로 배웠다.
-

2026-07-01 · 최종 인덱스 · 종합 점수 도입 · 튜닝/검증 분리

오늘 할 일

- 원본 인덱스 재빌드, 설정별 비교, 종합 점수와 증거 자료 정리

한 일 / 결과

- 원본 인덱스 재빌드 성공. 이전 문제들은 재발하지 않았다.
- 답변이 빈 문자열로 나오는 케이스가 있었는데, 원인은 검색이 아니라 생성 모델이 토큰을 추론에다 써버린 것이었다. 토큰을 올리고 빈 답변이면 재시도하게 수정.
- 여러 지표를 하나로 묶은 종합 점수(EDD)를 정의했다. 커버리지, 정답 순위, 근거성, 관련성, 거절 정확도, 속도를 가중합하고, 잘못된 거절과 빈 답변은 감점하는 방식.
- 설정 비교 결과, 원본 텍스트에 자동필터와 쿼리 재작성을 켜고 검색 후보를 8개 정도 주는 조합이 제일 좋았다. 종합 점수 96점대에 커버리지와 관련성은 거의 만점. 이 조합을 기본값으로 고정했다.
- 필터와 재작성이 서로 다른 실패를 보완한다는 것도 확인했다. 필터를 끄면 기관명을 명시한 질문이 깨지고, 재작성을 끄면 후속질문이 깨진다. 둘 다 켜야 한다.
- 보고서에서 바로 인용할 수 있게 질문, 답변, 근거, 지표를 한 묶음으로 정리한 증거 패킷도 만들었다.

고민 / 실수

- 거절 판정이 생각보다 까다롭다. "없는 요구사항을 없다고 답하는 것"과 "문서를 못 찾아서 거절하는 것"은 다른데, 처음 판정 방식은 이 둘을 섞어서 정상 답변을 거절로 오분류했다. 판정을 고치고 재계산했다. 평가기가 틀리면 개선 방향도 틀어진다.
- 지표가 거의 만점에 붙기 시작했다. 시스템이 완벽하다는 뜻이 아니라 평가셋이 쉽다는 신호일 수 있다. 멀티홉, 비슷하지만 없는 사업, 애매한 기관명, 오타 섞인 질문처럼 더 어려운 문제가 필요하다.

한 일 (이어서) · 새 질문셋과 튜닝/검증 분리

- 새 질문 25개를 만들어 검수하고 고정했다. 튜닝용 18개와 검증용 7개로 나누고, 검증용은 최종 확인 전까지 개선 작업에 쓰지 않기로 했다.
- 답변 모델과 채점 모델도 분리했다. 자기 답변을 비슷한 모델이 다시 칭찬하는 구조면 점수가 부풀 수 있어서.
- 튜닝셋 첫 측정은 80점 언저리. 오히려 좋은 신호였다. 전처럼 포화되지 않고 개선할 지점이 보였다. 특히 검색은 잘 되는데 답변이 근거에 없는 세부사항을 덧붙여서 근거성이 깎이는 문제.
- 그런데 뜯어보니 더 큰 원인은 평가기였다. 심판이 검색 근거의 앞부분 위주로만 보고 뒤쪽 근거를 못 본 채 "근거 없음"으로 깎고 있었다. 근거를 균등하게 보도록 고치자 점수가 크게 회복됐다. 모델을 고치기 전에, 평가기가 근거를 제대로 보고 있는지부터 확인해야 한다.

- 비교 질문에서 기관이 빠지는 문제는 기관 별칭 처리와 기관별 최소 근거 보장으로 보완했다. 보정을 합치니 튜닝셋이 95점대까지 올라왔다.
- 검증셋을 돌리기 전에 스스로 반박부터 적었다. 채점 방식이 후해진 효과일 수도 있고, 같은 질문셋에 맞춰진 결과일 수도 있다고.
- 실제로 검증셋 첫 결과는 81점대로 떨어졌다. 튜닝셋 점수가 그대로 일반화되지 않은 것. 실패 원인을 고치고 다시 재면 96점대까지 회복되지만, 이건 실패를 본 뒤 같은 문항을 다시 쟀 값이라 라벨을 달리 붙여야 한다.

소감

- 오늘은 숫자를 올리는 것보다, 숫자를 믿어도 되는지 확인하는 장치를 만드는 데 가까웠다. 이제 보고서에 "성능이 올랐다"만 쓰는 게 아니라, 어떤 질문에서 좋아졌고 어떤 질문은 왜 조심해야 하는지까지 말할 수 있게 됐다.

2026-07-02 · 점수 라벨 정리 · 새 질문셋 루프 · 채점기 의심하기

오늘 할 일

- 어제 나온 점수들이 각각 뭘 의미하는지 정리 (뭘 믿어도 되는 숫자인지)
- 시간 기준선을 두고 개선 루프 돌려보기

한 일 / 인사이트

- 점수 라벨부터 바로잡았다. 튜닝셋 95점대, 첫 검증셋 81점대, 실패를 보고 고친 뒤 재측정이 96점대. 마지막 값은 같은 문항을 다시 쟀 것이라 검증 성능이라고 쓰면 안 된다. 보고서엔 81점대를 일반화 근거로 쓰고, 재측정값엔 꼬리표를 달기로.
- 오타, 후속질문, 없는 정보 우기기 같은 걸 섞은 새 검증 12문항을 만들어 돌렸다. 첫 실행은 90점 언저리였는데, 답변은 제대로 거절했는데 판정식이 그 표현을 못 알아본 문항이 있었다. 판정만 고쳐 재계산하면 96점대. 이 둘도 라벨을 나눠 적었다.
- 검색 후보 수를 줄이면 2~3초 빨라지는데 결제 연동 같은 세부 근거를 놓친다 → 기각. 원문을 직접 뒤져보니 근거는 문서 안에 분명히 있었고, 후보가 적을 때 그 조각을 못 가져온 거였다.
- "안양"처럼 도시명만 짧게 부르면 기관 필터가 안 걸리는 문제를 별칭으로 보정했다. 그랬더니 이번엔 안양대학교, 안양천까지 잡는 오탐이 생겨서 예외 처리를 추가. 보정 하나 하면 반대쪽 부작용을 꼭 확인해야 한다.
- 채점기를 그냥 믿으면 안 되겠어서, 일부러 틀린 답변을 심은 검증팩을 만들어뒀다. 나중에 채점기가 이걸 못 잡으면 높은 점수도 착시라는 뜻.
- 대표 답변 몇 개를 사람 눈으로 리뷰해보니 5점 만점에 3점대 중반 수준. 자동 지표는 좋아 보여도 답변이 길고 나열식이라 아직 멀었다.

고민

- 점수가 오르면 진짜 개선인지, 측정 보정인지, 과적합인지 계속 헷갈린다. 그래서 첫 실행 점수, 재계산 점수, 진단용 결과를 분리해 기록하는 규칙과 질문셋별 노출 여부 표를 만들었다. 한 번 노출된 셋의 점수는 회귀 확인용으로만 쓴다.

소감

- 오늘은 숫자를 올린 날이 아니라 숫자를 믿을 수 있게 만든 날. "어떤 숫자는 못 믿게 됐는가"까지 적어두는 게 다음 개선의 출발점이다.

2026-07-03 · 채점기 검증 · 새 검증셋 첫 점수 · 적대 질문

오늘 할 일

- 비용 한도 안에서 채점기 검증부터, 그 다음 동결해둔 질문셋 첫 실행
- 실제 사용자가 던질 법한 못된 질문 만들기

한 일 / 결과

- 채점기 검정부터 했다. 점수 범위가 한 번 흔들려서 범위를 강제하고 벗어나면 실패로 세계 보강. 검정 문항 하나는 문항 자체가 애매했던 거라 더 엄격한 묶음으로 교체하고 전부 통과시켰다.
- 동결해 둔 검증셋 첫 실행은 97점대. 오늘의 기준점이다. 다만 평균 20초 가까이 걸려서 속도는 아쉽다.
- 속도와 형식 후보를 여럿 시험했는데 전부 기각했다. 검색 후보를 줄이면 이득은 1초도 안 되는데 답변이 흔들리고, 보고서용 형식은 보기엔 좋은데 응답이 30초를 훌쩍 넘겼다.
- 적대 질문셋(사업명 조각만 던지기, 낙찰가 추정 요구, 개인정보 요구 등)을 만들어 돌렸더니 92점대로 떨어졌다. 사업명 조각만 있으면 기관 필터가 안 켜져서 엉뚱한 문서가 섞이는 약점이 드러난 것 → 제목 조각 매칭 필터를 추가해 해결. 보정 후 재계산은 98점대까지 나왔지만, 노출된 셋 재측정이라 성능 주장엔 못 쓴다.
- 기존 검증셋 회귀 확인: 새 필터를 넣어도 안 깨졌지만, 기존 기준점을 넘지도 못했다. 검색 후보 축소의 전역 채택은 다시 기각.

소감

- 점수가 높을수록 더 의심해야 한다. 실패를 보고 고친 값은 개선 과정의 증거지, 처음 보는 질문에 서의 성능 증거가 아니다.

2026-07-04 · RAG 평가 방법 조사

한 일 / 인사이트

- 기존 RAG 평가 프레임워크와 논문 자료를 훑어봤다. 결론은, 질문을 더 많이 만드는 것보다 "틀리는 방식을 강제로 드러내는 평가 장치"를 정교하게 만드는 게 다음 개선에 맞다는 것.
- 다음 루프 후보 네 가지를 정리했다. 같은 뜻의 변형 질문에서 답이 흔들리지 않는지 보는 테스트, 정답 근거를 빼거나 가짜 근거를 넣어보는 테스트, 답변을 주장 단위로 쪼개 각 주장마다 근거가 있는지 세는 감리, 그리고 작은 점수 차이는 개선으로 치지 않는 기준.
- 이 원칙들은 이번 과제 이후에도 계속 쓸 것 같아 따로 정리해 저장해뒀다.

소감

- 점수가 거의 천장이라, 이제 소수점 상승보다 실패 유형이 줄었는지, 사람 기준 품질이 좋아졌는지가 중요하다.

2026-07-05 · 변형 질문으로 천장 깨기 · 안전 경계 보강

오늘 할 일

- 시스템의 성질을 흔드는 변형 질문으로 다시 약점 찾기

한 일 / 결과

- 말투 변형, 너무 일반적인 제목 조각, 낙찰가와 개인 연락처 요구 같은 걸 섞은 새 질문셋을 고정하고 첫 실행. 86점대로 떨어졌는데, 이 하락이 오히려 수확이었다. 검색과 근거는 좋은데 거절 정확도가 바닥 — 안전 경계가 흔들리고 있었다.
- 사업 예산을 실제 계약금액처럼 다루는 답변을 발견 → 예산과 계약 결과를 분리하고, 공식 연락처와 개인 연락처를 분리하는 지침을 추가. 다시 확인해보니 위험한 부분은 좋아졌다.
- 같은 사업을 흐트러진 말투로 물으면 답이 달라지는 문제도 있었다. 원인은 모델 변덕이 아니라 기관 약칭이 정식 기관명으로 안 묶여 생긴 검색 오염 → 별칭 추가로 회복. 다만 출입통제 관련 근거는 원문에 있는데도 답변이 못 끌어쓰는 문제가 남았다. 검색 후보를 늘려봐도 기관을 잘못 잡은 상태에선 노이즈만 늘다는 것도 배웠다.
- "모호하다고 거절해놓고 뒤에 후보 설명을 길게 붙이는" 답변을 별도 품질 이슈로 잡기 시작했다. 거절 정확도가 맞아도 거절 뒤에 너무 많이 말하면 안전한 답이 아니다.
- 문항당 비용과 응답 시간을 기록하는 장부를 붙였다. 느린 원인의 대부분은 검색이 아니라 답변 생성이었다.
- 채점기가 부족한 답변에 만점을 주는 장면을 실제로 목격했다. "점수는 높는데 납득이 안 되는 답변"의 증거로 보고서에 꼭 넣을 것.

소감

- 이 질문셋의 정직한 첫 점수는 86점대로 기록한다. 그 뒤 수치는 전부 실패를 보고 고친 재시도다. 다음 핵심은 주장 단위 감리.

2026-07-06 · 비용 안전장치 · 과잉응답 게이트

한 일

- 유료 루프를 더 열고 싶은 유혹이 있었지만 멈추고 안전장치부터. 실행 전에 예상 비용이 한도를 넘으면 아예 시작하지 않는 장치를 붙이고, 일부러 작은 한도를 걸어 실제로 멈추는지 확인했다.
- 스스로 비판적으로 다시 보니 검증 방법이 틀려 있었다. 원래 돈이 안 드는 시험 실행이 예산에 걸려 멈춘 건 증거가 아니다 → 무비용 경로와 유료 경로를 분리하고, 비용 기록은 생기는 즉시 장부에 남기도록 고쳤다.
- 과잉응답 감지도 보강했다. 길이만 보지 않고, 짧아도 예산이나 일정 같은 세부 항목을 여러 개 붙이면 잡히게.

소감

- 점수는 안 올랐다. 대신 다음에 점수가 올랐을 때 그 숫자를 믿기 위한 장치가 정확해졌다.

2026-07-07 · 주장 보존 게이트 · 밤샘 안정화 · 거절 경계 · 질문은행 정리

오늘 할 일

- 채점기 점수와 사람 눈의 간극(핵심 주장 누락) 잡기
- 속도와 거절 경계 다듬고, 새로 받은 질문은행 정리

한 일 / 결과

- 주장 보존 게이트를 만들었다. 채점기가 만점을 준 답변도 사람이 보면 핵심 주장이 빠져 있는 경우가 있어서, 저장된 답변에서 기대한 주장이 살아남았는지 자동으로 확인한다. 근거 대조는 해당 기관과 사업 범위 안으로만 좁혀서 그럴듯한 근거를 엉뚱한 데서 빌려오지 못하게 했다.
- 밤샘 루프. 핵심 주장이 빠진 진짜 원인은 답변이 아니라 문맥이었다. 필요한 문구가 요약 데이터에만 있어서, 기관과 사업명이 일치할 때만 요약을 보강하게 했다. 같은 기관의 다른 사업 내용이 섞이는 문제엔 사업명 초점 필터를, 모순되거나 중복된 문장엔 정리 규칙을 더했다. 노출된 셋 회귀는 97점대로 안정화됐지만 이건 회귀 안정화지 새 검증이 아니다. 보고서용 근거는 여전히 첫 실행의 86점대.
- 속도 실험. 프롬프트를 간결하게 하면 1초도 못 아끼면서 거절 능력을 깨먹어서 기각. 검색 후보를 늘리는 것도 근거성이 떨어져 기각. 결국 현행 유지가 결론. 조금 빠른 오답은 개선이 아니다.
- 질문셋 상태 기록에서 오염을 발견했다. 이미 써버린 질문셋이 미사용 후보로 잘못 남아 있었고, 그대로 새 검증이라고 썼으면 가짜 검증이 될 뻔했다. 평가셋의 상태 기록 자체도 측정 장치라서, 이게 낡으면 숫자가 거짓말한다.
- 거절 뒤 꼬리 문제. 거절은 하는데 뒤에 세부 설명과 연락처를 붙이는 답변을 플래그로 잡고, 애초에 지어내야만 답할 수 있는 질문(낙찰가, 환자 사례, 개인 연락처 조작 요구)은 생성 전에 짧게 거

절하게 했다. 민감 문항이 30초 가까이 걸리던 게 1초 미만으로. 대신 "추정금액" 같은 정상적인 조달 용어까지 막지 않도록 조건을 계속 좁혔다.

- 필드 단위 채점. 한 질문 안에 답해야 할 것(예산, 평가기준, 공식 문의처)과 멈춰야 할 것(계약금액, 최종 점수, 개인 연락처)이 같이 들어온다. 전체를 거절 하나로 보면 계속 오판해서, 필드별로 과잉 거절과 위험한 누출을 따로 잡는 채점기를 만들었다.
- 질문은행 정리. 새로 받은 95문항 중 이미 여러 번 본 패턴을 걷어내고 68개를 남겼다. 성격이 다른 시험지들이라 하나의 점수로 섞으면 안 돼서 네 묶음으로 나누고, "이 사업" 류 질문엔 기준 사업을 고정하고, 집계형 문항은 별도 채점 경로로 빼고, 검색 기록이 있어야만 썰 수 있는 문항은 준비될 때까지 보류로 정직하게 표시했다. 실행 계획까지 굳혀놓고 답변 실행은 아직 안 했다.
- 빈 필드 진단. 요약 데이터 기준으로는 비어 보이던 필드가 원문에는 다 있었다. "문서에 없음" 판정이 모델 실패가 아니라 근거 원천을 잘못 고른 탓일 수 있다는 발견. 원문 기준으로 답할 수 있는 필드와 없는 필드가 섞인 문서를 다시 골라 진단 문항을 고정하고 첫 진단 실행까지 마쳤다.

고민 / 실수

- 긴 실험을 한 번에 돌리다 시간 초과로 결과가 통째로 날아간 적이 있다. 이후로는 작은 단위로 나누고 즉시 저장하는 방식으로만.
- 이번 주 내내 모델 실패보다 평가기 오탐이 더 자주 나왔다. 거절 감지 누락, 기대값 파일 인코딩 깨짐, 게이트가 설명문 속 단어에 반응하는 문제까지. 평가 장치도 평가해야 한다.

소감

- 좋은 답은 보이는 건 답하고 없는 건 없다고 말하는 것이고, 좋은 평가는 그 차이를 구분하는 것. 점수 올리기보다 "어떤 숫자를 보고서에 쓰면 안 되는지"가 훨씬 선명해진 한 주였다. 다음은 필드 단위 기대를 붙인 진단 재실행과, 준비해둔 질문은행 실행이다.